

Reading Nested JSON Objects from Azure Cosmos DB using .NET SDK

Summary:-

In this lab, you will use the Azure Cosmos DB .NET SDK to read a document containing nested JSON objects from an Azure Cosmos DB container. You will retrieve the document using its **ID** and **Partition Key**, then display both the top-level properties and the nested **Course** and **Payment** object properties.

Let's Start:-

After completing this lab, you will be able to:

- Read a document from Azure Cosmos DB using the .NET SDK.
- Retrieve a document using its **ID** and **Partition Key**.
- Access nested JSON objects using .NET classes.
- Display nested object properties using the dot (.) operator.

Lab Steps:-

Step 1 – Prepare the Azure Cosmos DB .NET Project

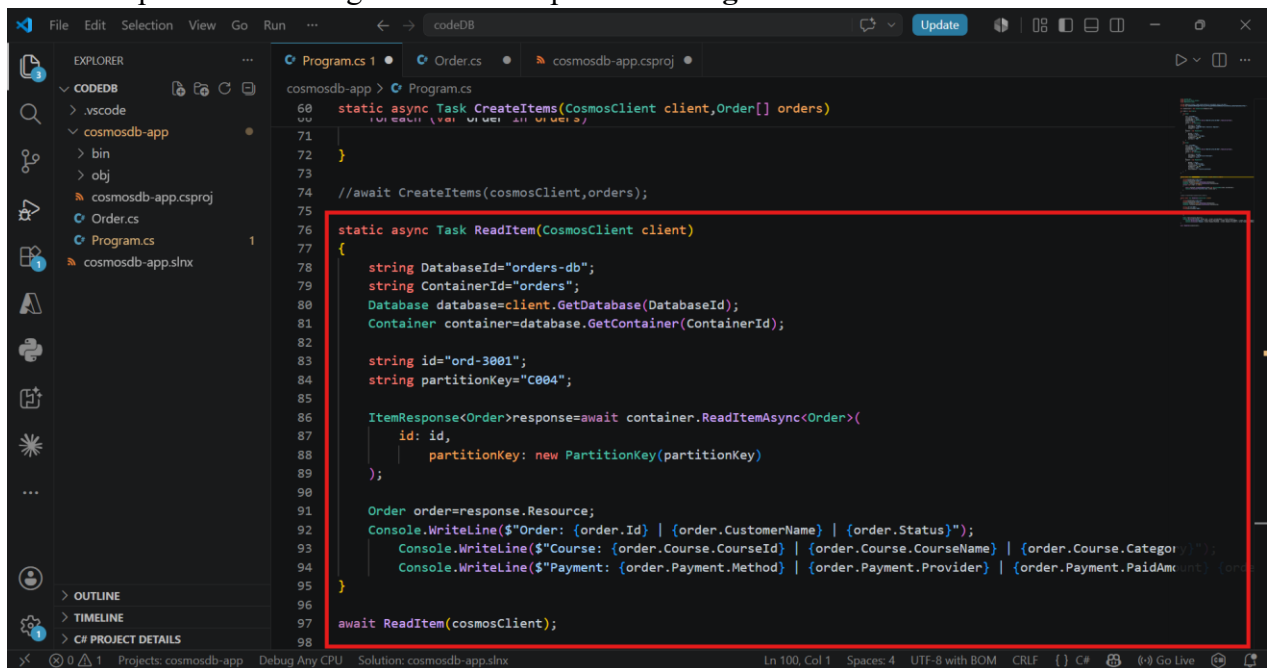
- **Reference:** This lab uses the .NET console application, Azure Cosmos DB account, database, and container created in the previous lab.
- For that **create** Azure Cosmos DB account by selecting **Azure Cosmos DB for NoSQL** from the recommended APIs.
- Configure:
Workload type: Learning, **Region:** EAST US/ EAST US 2, **Capacity Mode:** Provisioned Throughput
- Enable **Apply Free Tier Discount**. Click **Next**.
- Ensure Geo-Redundancy and Multi-region Writes is **Disabled**. Click **Next**.
- Select **Enable access from all networks**. Click on **Review + Create** to create.
- From the left pane, select **Keys**.
- Copy: **URI** and **Primary Key**
- Open **Visual Studio Code**. Create a new project folder.
- Press **Ctrl + Shift + P** to open the **Command Palette**.
- Select **Show and Run Commands > .NET: New Project > Console App**(Rename it as cosmosdb-app) > **Default Directory > .slnx > Create Project**.
- **Open** the integrated terminal. Run:

```
cd .\cosmosdb-app\  
dotnet add package Microsoft.Azure.Cosmos  
dotnet add package Newtonsoft.Json
```
- Create a new class file named **Order.cs**.

- Open the following project files:
Program.cs
Order.cs
cosmosdb-app.csproj
cosmosdb-app.slnx
- Replace the existing content with the **provided files**.
- In **Program.cs**, **Replace Endpoint with URI** and **Key with Primary Key**.
- **Save** all the files and **run**:
dotnet run
dotnet build
- This step creates a .NET console application and prepares it to insert items into the Azure Cosmos DB container.

Step 2 – Update the Application Code

- Open **Program.cs**.
- Replace the existing code with the provided **Program.cs** file.



```

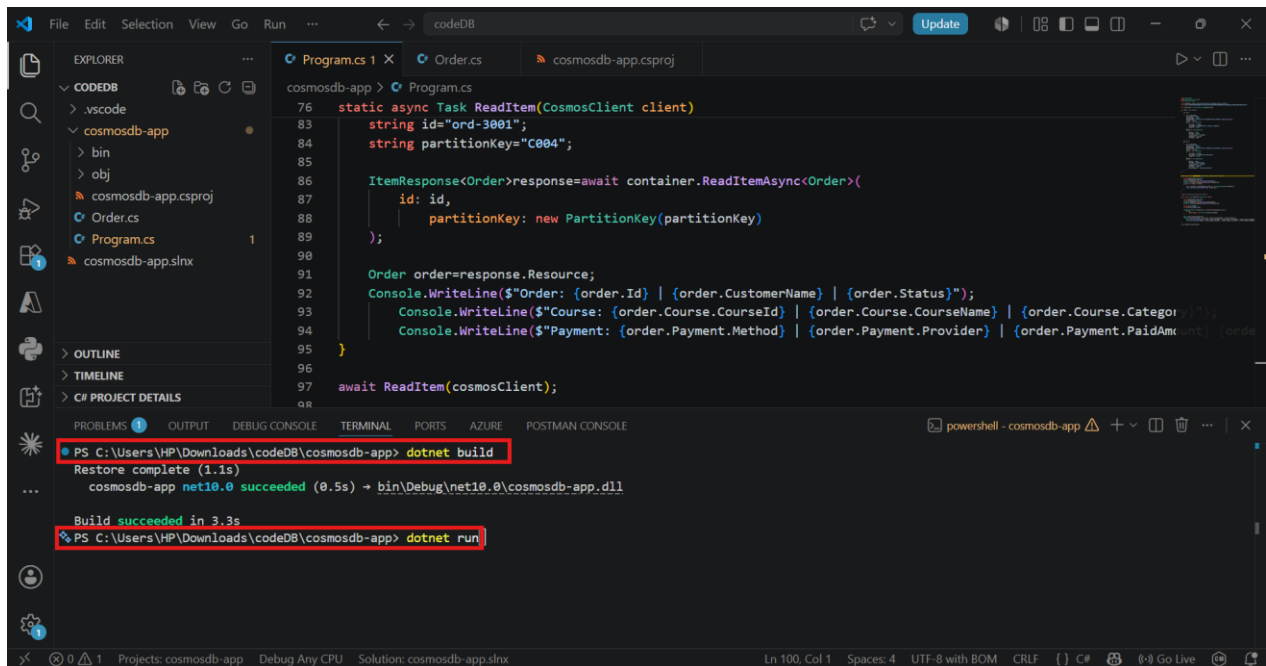
60 static async Task CreateItems(CosmosClient client, Order[] orders)
61 {
62     foreach (var order in orders)
63     {
64         await client.CreateItemAsync<Order>(order, new PartitionKey(order.Id));
65     }
66 }
67
68 //await CreateItems(cosmosClient, orders);
69
70 static async Task ReadItem(CosmosClient client)
71 {
72     string DatabaseId="orders-db";
73     string ContainerId="orders";
74     Database database=client.GetDatabase(DatabaseId);
75     Container container=database.GetContainer(ContainerId);
76
77     string id="ord-3001";
78     string partitionKey="C004";
79
80     ItemResponse<Order>response=await container.ReadItemAsync<Order>(
81         id: id,
82         partitionKey: new PartitionKey(partitionKey)
83     );
84
85     Order order=response.Resource;
86     Console.WriteLine($"Order: {order.Id} | {order.CustomerName} | {order.Status}");
87     Console.WriteLine($"Course: {order.Course.CourseId} | {order.Course.CourseName} | {order.Course.Category}");
88     Console.WriteLine($"Payment: {order.Payment.Method} | {order.Payment.Provider} | {order.Payment.PaidAmount}");
89 }
90
91 await ReadItem(cosmosClient);
92

```

- **Save** the file.
- This step updates the application to read a document containing nested JSON objects from the Azure Cosmos DB container.

Step 3 – Build and Run the Application

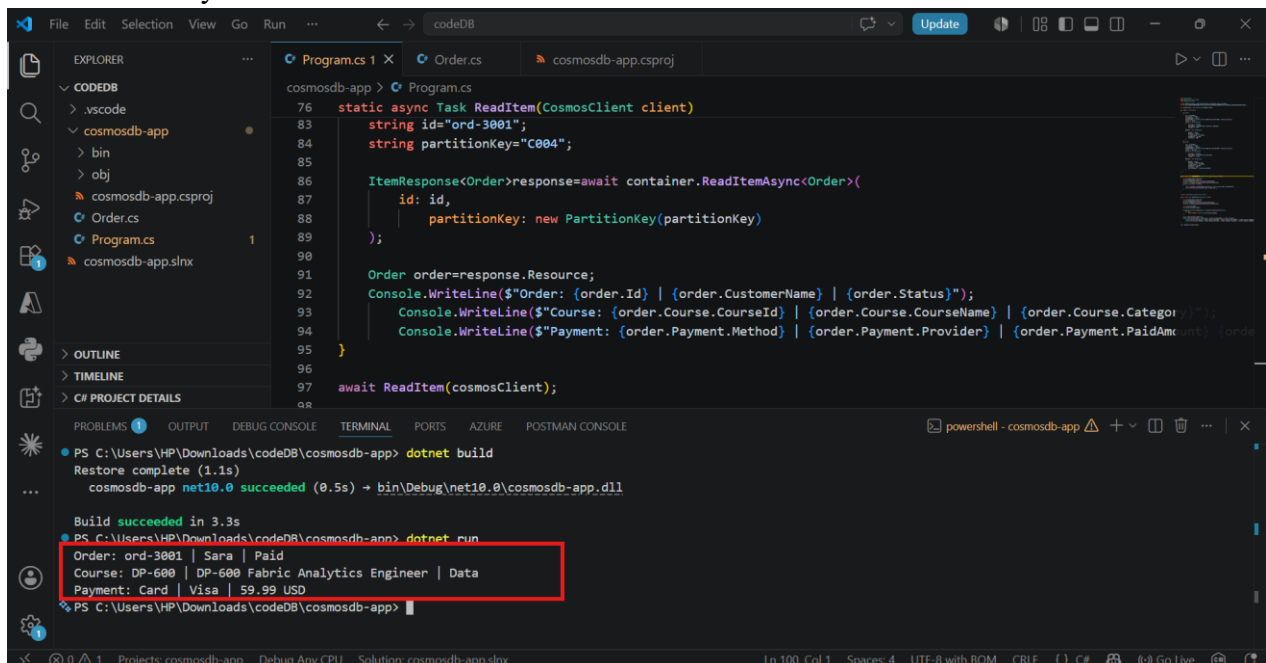
- **Open** the integrated terminal and **run**:
dotnet build
dotnet run



- Wait for the application to display the document details.
- This step reads the specified document from the Azure Cosmos DB container.

Step 4 – Verify the Output

- **Observe** the console output.
- **Verify** that the order details, course information, and payment details are displayed correctly.



- This step confirms that nested JSON object properties are read successfully from Azure Cosmos DB.

Verification

Verify the following:

- The application builds and runs successfully.
- The document is retrieved using the correct **ID** and **Partition Key**.
- The top-level order details are displayed correctly.
- The nested **Course** and **Payment** object properties are displayed correctly.
- No errors occur while reading the document.